

Recommender System Challenge

Recommender Systems Assignment 3

Yesmina El Arkoubi: S2989018

Manuel Pérez Belizón: S4416880

Gaurisankar Jayadas: S4374886

6 April 2025

1 Introduction

Recommender systems are crucial for today's digital platforms, facilitating personalized content discovery in various fields like e-commerce, entertainment, and social media. With the increasing scale and intricacy of user-item interactions, developing precise and efficient recommender systems has emerged as a significant research and engineering challenge.

This project focuses on building an end-to-end recommender system that effectively combines predictive accuracy with strong engineering principles. The foundation for model training and assessment is a consolidated user-item interaction dataset, with performance evaluated through ranking-based metrics applied to a hidden test set. The models will be compared against each other using a public leaderboard hosted on Kaggle, where **Recall@10** serves as the main evaluation metric.

In addition to performance, this project highlights the importance of reproducibility and analytical rigour. We record the entire process, from data cleaning and model development to hyperparameter tuning and failure analysis, in a comprehensive technical report. The aim is not simply to secure top positions on leaderboards but also to showcase a deep understanding of recommendation systems and the inherent trade-offs involved.

Through this challenge, we aim to synthesize knowledge from collaborative, sequential, and neural recommender models, and develop a system that is both effective in prediction and robust in design.

2 Theory

2.1 Bert4REC

The first model of this assignment is built on the BERT4Rec framework [sun2019bert4rec] utilizing transformer-based masked item prediction for sequential recommendation tasks. The design draws inspiration from BERT in NLP, facilitating bidirectional learning across user-item interaction sequences.

2.1.1 Sequential Recommendation

Sequential recommendation emphasizes forecasting the next item a user will likely engage with, drawing from their previous interactions. It takes the sequence of actions into account, unlike conventional collaborative filtering that views all past activities on the same level.

2.1.2 BERT4Rec Architecture

BERT4Rec is a sequential recommendation model based on the Bidirectional Encoder Representations from Transformers (BERT) architecture utilized in natural language processing. Unlike conventional unidirectional models that only account for prior user interactions, BERT4Rec captures bidirectional context within user behavior sequences by employing a masked item prediction objective during training. This structure allows it to more effectively represent the intricate dependencies in item sequences and generate precise next-item predictions.

The model handles item sequences like word sequences in NLP. In a sequence of user interactions, specific items are randomly masked, and the model is trained to predict these masked items based on the surrounding context. This approach enables BERT4Rec to develop comprehensive, context-sensitive embeddings of user preferences.

2.1.3 Transformer Encoder Blocks

The core of the BERT4Rec model consists of stacked transformer encoder layers, each comprising two primary components:

- **Multi-Head Self-Attention:** The attention heads empowers the model to focus on various aspects in a sequence, allowing it learn intricate relationships, with each head containing its own Q, K, V matrices. Therefore, the higher the number of attention heads, the more different features the model can capture. The self-attention mechanism facilitates bidirectional interaction, crucial for understanding context from both previous and forthcoming interactions.

- **Feedforward Layers:** Each attention output is passed through a position-wise feedforward network, which consists of linear transformations and non-linear activations. These layers increase the capacity of the model to capture high-level abstract patterns in the data.

Each encoder layer incorporates residual connections and layer normalization to ensure stable training and effective gradient flow.

2.1.4 Masked Language Modeling

Masked Language Modeling (MLM) is a training strategy where a portion of tokens in an input sequence are randomly replaced with a special (i.e. masked) token. The model's objective is to predict these masked tokens based on the context provided by the unmasked tokens. This is ideal for a BERT model as it forces the model to learn deep bidirectional representations of sequences, as it must leverage information from both the left and right of the masked tokens to make accurate predictions.

2.2 Neural Collaborative Filtering

The second model implemented in this assignment is based on a Neural Collaborative Filtering (NCF) model. NCF models utilize deep learning to enhance collaborative filtering by learning intricate, non-linear functions that represent user-item interactions.

This is done by using an MLP layer that captures non-linear interactions between users and items. Rather than calculating a product of embeddings, MLP combines user and item embeddings, then processes them through several fully connected layers utilizing non-linear activation functions like ReLU.

2.3 The cold-start problem

One of the fundamental challenges in recommender systems is the cold-start problem, where the system has limited or no interaction data for new users or items. Traditional models usually depend on the availability of past user-item interactions to identify patterns of preference. In the absence of such data, these models cannot effectively compute user similarities or infer preferences, leading to poor recommendation performance. Addressing the cold-start problem typically requires incorporating external sources of information, such as item meta-data or user profiles, to enrich representations and enable meaningful recommendations despite limited interaction history. Therefore, to address this challenge, in both the Bert4REC and the NCF algorithm we included the meta data as embeddings to enhance the performance.

2.4 Negative Sampling

Negative sampling is a strategy used to simulate negative feedback by randomly selecting items that a user has not rated or interacted with. These items are treated as negative examples during training or evaluation. The core assumption is that the user is less likely to prefer

these unobserved items compared to the ones they have explicitly interacted with.

2.5 Re-ranking Evaluation Strategy

The re-ranking strategy is a technique used in recommendation systems and other ranking tasks to address the challenge of large item vocabularies. Instead of computing scores for all possible items (which can be computationally expensive), the model tries to distinguish between positive interactions (ground truth) and negative interactions (unseen). During evaluation, the model creates a candidate set that includes the future items (ground truth) along with several randomly selected negative items, the model then predicts scores for all candidates, and the candidates are ranked according to these scores.

This approach reduces computational cost and focuses more on the ranking quality rather than absolute accuracy, measuring how well the model can predict relevant items.

3 Methodology

3.1 Data Preprocessing

The interaction dataset includes records of implicit feedback in the form of clicks or views. The dataset used for training consists of 346612 user-item interactions, including 323625 unique users and 65295 items. The dataset is very sparse, containing 1 interaction per user for 94.60% of the users as shown in Figure 1. Also, the train dataset contains 3462 duplicates and the test set 14 duplicates, which are removed to reduce bias.

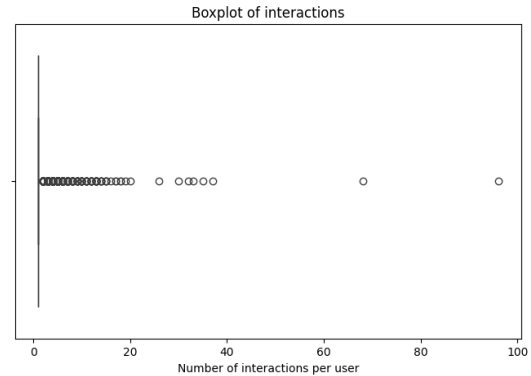


Figure 1: Boxplot showing the distribution of the number of interactions per user in the training set. The average number of interactions is 1.07 as 94.60% of the users only have 1 interaction

3.2 Item Metadata Processing

Apart from the user-item interaction dataset, we have an item metadata dataset, that contains very useful information like item descriptions, prices, categories or average ratings. Because of the sparsity of the dataset and the informativeness of the metadata we decided to incorporate some features. More specifically, we use:

- **Title embeddings:** Derived from TF-IDF features of item titles, resulting in 100-dimensional representation
- **Categorical embeddings:** Encoded from item categories.
- **Rating scores:** (Min-Max)Scaled numerical features representing average item rating.

These additional features are combined with the models to create a hybrid recommender system that integrate collaborative and content-based filtering.

3.3 Bert4REC

3.3.1 Masking Strategy

Throughout the training process, sequences were limited to a maximum length of 10. Items were randomly masked with a probability of 15%. The masked positions retained their original item ID while all other positions were marked with -100, which the loss function disregards.

3.3.2 Model Architecture

The model contains the following components:

- **Positional Embeddings:** Maintain the sequential order of items, which is vital for recognizing temporal patterns in user behavior.
- **Item Embeddings:** Maps each item to a learnable dense vector representation.
- **Transformer Encoder:** A stack of 4 transformer blocks, each consisting of a multi-head self-attention, receives the positional and item embeddings and outputs a contextualized representation for each token(i.e. item) in the sequence. layer with 4 different attentions heads
- **Feature Fusion:** Concatenates transformer outputs with title TF-IDF, category embeddings, and rating values.
- **Final Prediction Layer:** A linear layer projects the fused representation to logits over all items.

3.3.3 Training Setup

The model is trained with the MLM objective, which means it randomly masks 15% of the items in the sequence with a special token -100 and then predicts these items. The model learns by computing the cross entropy loss of the predicted and target items, which is then used to update the network weights. Adam is used as the optimizer with a learning rate of 0.001.

3.3.4 Evaluation

During validation and testing, every item in the sequence is masked and added to a candidate set along with 100 negative random samples.

The aim is that the model ranks the true future items highest(i.e. within the top 10) among the candidates for each sequence.

The model performance is assessed using NDCG@10 (Normalized Discounted Cumulative Gain), which gives higher scores to relevant items ranked higher in the predicted top-10 list. This metric considers both relevance and ranking position, making it ideal for evaluating recommendation quality.

3.4 NCF

3.4.1 Model Architecture

The model contains the following components:

- **User Embeddings:**Users represented as a learnable dense vector.
- **Item Embeddings:**Items represented as a learnable dense vector.
- **Features Embeddings:**Features embeddings for title, categories and average ratings.
- **MLP layer:** Fully connected layers using ReLu activation function that receive the concatenated embeddings and features
- **Output layer:** Final dense layer that uses a sigmoid activation function, resulting in the predicted probability of user-item interactions.

3.4.2 Training Setup

The NCF model is trained using user-item interactions, along with item metadata. The model trains with a negative sampling setup with 10 negative items for each positive user-item interaction, using binary classification on positive and negative interactions. Due to the sparsity of the train data, item metadata is also passed as embedding to the model to improve performance, like the aforementioned embeddings for the Bert4REC model. The user, item, and metadata embeddings are passed to the model to make accurate predictions. Adam is used as the optimizer with a learning rate of 0.001 once again.

3.4.3 Evaluation

After training, the model is evaluated on the test set by computing the NDCG@10 for each user. This is done by ranking one positive item among 99 randomly sampled negative items.

4 Results

As part of the Kaggle competition, we were tasked with generating recommendation predictions for a specific subset of users in the test set. We evaluated both of

our hybrid models, Neural Collaborative Filtering (NCF) and BERT4Rec, on this subset, expecting BERT4Rec to outperform due to its sequence modeling capabilities. However, to our surprise, the hybrid NCF model achieved a recall@10 of 0.019 while BERT4Rec achieved a recall@10 of 0.0123, as measured by the competition’s official recall-based metric.

However, when evaluating both models on the entire test set, we observed a very interesting finding:

- BERT4Rec achieved an NDCG@10 of 0.45 as shown in Figure 2
- NCF achieved an NDCG@10 of only 0.24 as shown in Figure 3

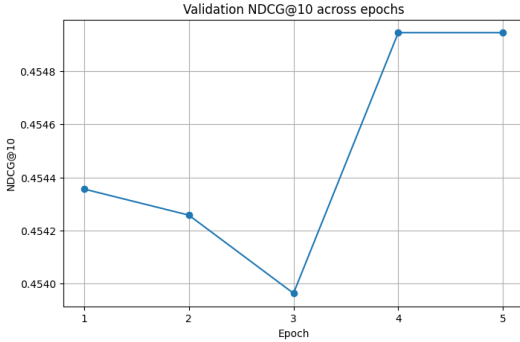


Figure 2: Validation across epochs for BERT4Rec model.

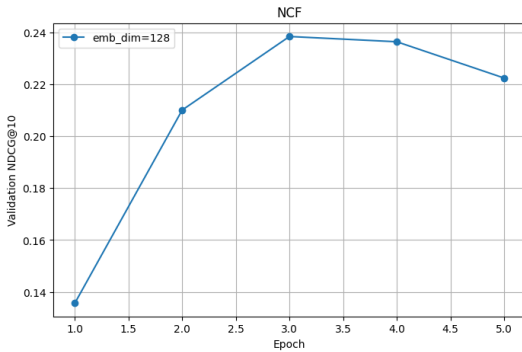


Figure 3: Validation across epochs for NCF model.

4.1 Hyperparameter tuning

To try and boost the tournament score of our NCF model we did a small hyperparameter tuning experiment. In particular, we tried three different values 32, 64 and 128 for the hyperparameter embedding size as this was the hyperparameter that most impacted our NCF in the previous assignment.

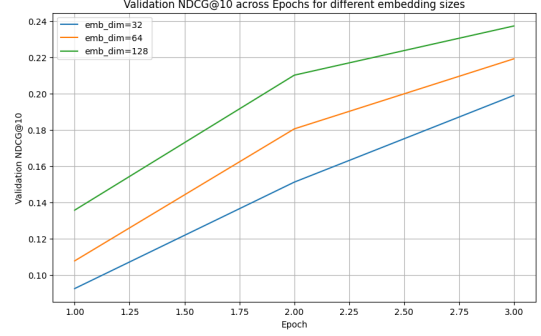


Figure 4: Validation across epochs for different embedding sizes on NCF model.

As shown in Figure 4 the biggest value offers the best performance this was also contrasted with a small boost on the tournament score where we went from recall@10 of 0.0190 to 0.0201.

5 Discussion

These results suggest that while BERT4Rec is more effective overall, its advantage may not fully materialize in limited or atypical test subsets like the one used for tournament evaluation. This leads to a few important observations:

- The small competition subset may have included a disproportionate number of cold-start users or users with minimal interaction history, where BERT4Rec’s reliance on sequence context is less effective. In contrast, NCF with metadata—can still generate meaningful recommendations even for users with sparse behavioral data.
- BERT4Rec benefits greatly from longer interaction sequences, therefore, underperforming in the tournament suggests short sequences for those specific users.
- BERT4Rec appears to generalize better on the full distribution of users, especially those with richer histories. On the other hand, NCF showed a good performance in cold-start or sparse settings due to its direct use of metadata, even if its overall performance is lower.

These findings gave us a critical takeaway: evaluation context matter. The perceived superiority of one model over another can change significantly depending on the characteristics of the test set. Therefore, the importance of evaluating recommender systems across diverse user profiles and varying data regimes. While, advanced sequential models like BERT4Rec offer superior performance in general, simpler architectures enriched with metadata may yield more reliable predictions in scenarios characterized by extreme sparsity or lack of historical user behavior.

6 Conclusion

In this project we implemented two hybrid recommender systems one based on BERT4Rec and the other based on NCF, extending both with rich item metadata to address the challenges of high sparsity and the cold-start problem. We integrated categorical embeddings, numerical ratings, and semantic representations of item titles to make informed predictions even in the absence of extensive user interaction histories. Our results demonstrate the trade-offs inherent in different modeling approaches. BERT4Rec, with its sequence-aware transformer architecture, achieved superior performance on the full test set, highlighting its strength in modeling rich user histories. In contrast, NCF, though generally less expressive, proved more effective on the competition subset, likely due to its robustness in cold-start scenarios where sequential data was limited or unavailable. These findings emphasize the importance of evaluation context in recommender systems. Different models may exhibit strengths in specific scenarios, for instance, hybrid NCF performed better in the cold-start-heavy competition subset, while BERT4Rec excelled on the full test set with richer user histories. This suggests that some models are inherently better suited to particular evaluation conditions, depending on factors such as user interaction sparsity or metadata availability.

Ultimately, this study reaffirms the value of combining collaborative and content-based filtering into hybrid recommender systems.